uses the knowledge we have on the cost function to update the solution.

- **Descent step**: Gradient descent is based on the geometric interpretation of cost functions; for derivable functions, we assume each solution to the problem can be interpreted as a point in an n-dimensional space, and a single step of gradient descent is performed by computing the gradient of the cost function at current point. Based on the gradient, we can update the nD point, and find a better solution.
- **Local minima**: These points, where the cost function takes a locally-optimal value, are the nemesis of optimization algorithms in general, and gradient descent in particular: the algorithm would get stuck in local minima and we would never find the globally-optimal solution.
- **Crossing number** doesn't play well as a cost function, because it makes a step function with plenty of local minima plateaus. As an alternative, we can map our problem into a so-called *force-directed* graph drawing simulation, which focuses on drawing a graph nicely, and optimizes crossing number only indirectly. The formulation of the cost function for these problems play nicely with gradient descent and other optimization algorithms.

---

[1] Unless somebody proves that P=NP, which is still unknown, but it's considered unlikely.

[2] This refinement process will also continue in the next chapters, introducing new categories of optimization algorithms.

[3] As we have seen in chapter 15, if we only draw edges as straight-line segments, for some graphs we won't be able to obtain an embedding with as few intersections as possible: that's why it's been introduced the rectilinear crossing number, the minimum number of edge intersections across all possible graph's straight-line drawings.

[4] Given a list of cities and the distances between each pair of cities, find the shortest possible route that visits each city exactly once and returns to the origin city. This is, needless to say, a NP-complete problem.

[5] Find the largest subset of vertices in a graph such that each vertex in the subset is adjacent to all the other vertices in the same subset. In other words, find the largest complete subgraph of a given